

Comparing Linearization Methods for Quantum Simulations of the Vlasov-Poisson Equations

Nicholas Soule-Albridge

University of Wisconsin-Madison, MSPQC

This paper aimed to get a resource estimation for solving the quantum Vlasov-Poisson equations, used in plasma physics modeling. I compare the approaches of two main papers: The first uses direct linearization to convert to a tridiagonal matrix system, and the second uses Carleman Linearization.

1 Introduction

The Vlasov equation was first introduced by Anatoly Vlasov in 1938. It is a fundamental model that is used to predict plasma dynamics and describe the evolution of the distribution function of a plasma. Unlike approaches like the Boltzmann equation which uses a kinetic approach, or magnetohydrodynamics (MHD) which averages over velocity, the Vlasov equation allows capturing of non-linear phenomena like Landau damping and wave-particle interactions.

This increased complexity comes with scaling problems due to the high-dimensionality and is difficult to solve fully for utility-scale applications. In practice the Vlasov equation is usually coupled with either the Maxwell or Poisson equations. Classical solvers for the Vlasov equation are used in astrophysics and fusion research, but are difficult to solve at the resolutions needed.

My goal for this work was to investigate how logical qubit count, T-gate complexity, and T-depth scale with phase-space resolution, simulation time, and target accuracy for fault-tolerant quantum simulation of linearized Vlasov dynamics, and to identify the parameters that most strongly influence these resource requirements.

2 Classical Approach

The general (dimensionless) Vlasov equation with collisions is usually coupled to the Poisson equation to find the electric field, so the system to solve is

$$\dot{f} + \mathbf{v} \cdot \nabla_x f + \mathbf{E} \cdot \nabla_v f = C[f] \quad (1)$$

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0} \quad (2)$$

In this paper, I will use the non-collisional case with no external magnetic field since this can be used to evaluate the Landau damping rate, which is a very well known phenomena to compare to.

2.1 Hermite Representation

While particle and grid based methods allow for faster execution, spectral methods for solving PDE's allow for higher accuracy and are more resistant to statistical noise, especially for long time-scales [3].

In Ameri et al., they instead linearize about a Maxwellian background F_0 so that $f = F_0 + g(x, v, t)$ by taking first order approximations for each term. This results with the system of equations [2]

$$\frac{dg_k}{dt} + ikvg_k + ikvF_0\phi_k = C[g_k] \quad (3)$$

$$\phi_k = \alpha \int_{-\infty}^{\infty} g_k dv \quad (4)$$

where they use a Lenard-Bernstein collision operator for the $C[g_k]$ term. If you do not include collisions, the resulting matrix system is a hermitian matrix and can be directly simulated on a quantum computer, but if you do include collisions you get the system $\frac{dg_k}{dt} = Ag_k$, $A = -iH + \nu D$. [2]

In Kormann et al., they instead use modified the Hermite functions,

$$H_l^{\gamma, \epsilon} = \frac{1}{\sqrt{2^l l!}} h_l(\gamma \epsilon v) e^{-\epsilon^2 v^2} \quad (5)$$

where $h_l(x)$ are Hermite polynomials. ϵ is just a scaling parameter, and γ is used to change between a symmetric and anti-symmetric basis, and each have their own tradeoffs in conserving mass, energy, L^2 norm, etc.

The position is expanded in a Fourier basis and velocity in this modified Hermite basis as:

$$f(x, v, t) = \sum_{k=-N_x}^{N_x} \sum_{l=0}^{N_v-1} c_l^k(t) \exp\left(\frac{2\pi i k x}{L_x}\right) H_l^{\gamma, \epsilon}(v) \quad (6)$$

Plugging this into the Vlasov-Poisson system gives a set of differential equations for these coefficients. [3]

$$\begin{aligned} \dot{c}_l^k(t) = & -\frac{1}{\sqrt{2}\gamma\epsilon} \frac{2\pi i k}{L_x} \left(\sqrt{l+1} c_{l+1}^k(t) + \sqrt{l} c_{l-1}^k(t) \right) \\ & + \frac{\sqrt{2}\epsilon}{\gamma} \left((\gamma^2 - 1) \sqrt{l+1} [\mathbf{E}(t) * \mathbf{c}_{l+1}(t)][k] - \sqrt{l} [\mathbf{E}(t) * \mathbf{c}_{l-1}(t)][k] \right) \end{aligned} \quad (7)$$

$$E_k(t) = \frac{iL_x}{2\pi k} \sum_{l=0}^{N_v-1} c_l^k(t) I_l \quad \{E_0 = 0\} \quad (8)$$

where $[E * c][k]$ is a convolution and I_l is defined recursively as

$$\begin{aligned} I_0 &= \frac{\sqrt{\pi}}{\epsilon}, \quad I_1 = 0 \\ I_{l+2} &= \sqrt{\frac{l+1}{l+2}} (\gamma^2 - 1) I_l \end{aligned} \quad (9)$$

The Carleman Linearization technique can convert a differential equation of the form $\frac{du}{dt} = F_0(t) + F_1(u) + F_2(u * u) \dots$ into a matrix system $\frac{du}{dt} = \mathbf{A}u$. Using this for Eq 7 we can find the matrix $\mathcal{A}$, which is done in the pyLIQTR example notebook "linearization_tools.ipynb" for a small system with $N_x = N_v = 4$. Even for this case, the resulting sparse matrix has around 12 million entries and 1.7 million rows/columns.

2.2 Converting to a matrix system

For the case first linearizing the equations, then expanding in the Hermite basis (as in Ameri et al.) the resulting matrix system is simply tridiagonal. However for equations (7) & (8), this approach is more complicated. The convolution term can be written as [3]

$$[\mathbf{E}(t) * \mathbf{c}_l][k] = \sum_{j=-N_x}^{N_x} E_{k-j}(t) c_l^j(t) \quad (10)$$

and plugging this and (8) into (7) gives

$$\begin{aligned} \dot{c}_l^k(t) = & -\frac{1}{\sqrt{2}\gamma\epsilon} \frac{2\pi i k}{L_x} \left(\sqrt{l+1} c_{l+1}^k(t) + \sqrt{l} c_{l-1}^k(t) \right) \\ & + \frac{\sqrt{2}\epsilon i L_x}{\gamma} \frac{1}{2\pi} \sum_{j=-N_x}^{N_x} \sum_{m=0}^{N_v-1} \frac{I_m}{k-j} c_m^{k-j}(t) \left((\gamma^2 - 1) \sqrt{l+1} c_{l+1}^j(t) - \sqrt{l} c_{l-1}^j(t) \right) \end{aligned} \quad (11)$$

this system does have $(2N_x + 1) * N_v$ coefficients but cannot be converted directly to a matrix equation like $\dot{c}_l^k(t) = \sum_{n,m} (\mathcal{A})_{n,m} c_m^n(t)$ since there are nonlinear c^2 terms. This is where the Carleman Linearization is used [4]. Writing (11) in this general Carleman framework $\dot{\mathbf{u}} = F_0 + F_1 \mathbf{u} + F_2 \mathbf{u}^{\otimes 2}$, or explicitly writing out the indices,

$$\dot{c}_l^k = F_0 + \sum_{a,b} (F_1)_{lk,ab} c_a^b + \sum_{a,b,c,d} (F_2)_{lk,ab,cd} (c_a^b c_c^d) \quad (12)$$

so the F tensors are

$$F_0 = \mathbf{0} \quad (13)$$

$$\begin{aligned} (F_1)_{lk,ab} = & -\frac{1}{\sqrt{2}\gamma\epsilon} \frac{2\pi i k}{L_x} \left(\sqrt{l+1} \delta_{a,l+1} + \sqrt{l} \delta_{a,l-1} \right) \delta_{b,k} \\ = & -\frac{\sqrt{2}\pi i k}{\gamma\epsilon L_x} \left(\sqrt{l+1} \delta_{a,l+1} + \sqrt{l} \delta_{a,l-1} \right) \delta_{b,k} \end{aligned} \quad (14)$$

$$\begin{aligned} (F_2)_{lk,ab,cd} = & \frac{\sqrt{2}\epsilon i L_x}{\gamma} \frac{1}{2\pi} \sum_{j=-N_x}^{N_x} \sum_{m=0}^{N_v-1} \frac{I_m}{k-j} \left((\gamma^2 - 1) \sqrt{l+1} \delta_{a,l+1} - \sqrt{l} \delta_{a,l-1} \right) \delta_{b,j} \delta_{c,m} \delta_{d,k-j} \\ = & \frac{i\epsilon L_x}{\sqrt{2}\pi\gamma} \frac{I_c}{d} \left((\gamma^2 - 1) \sqrt{l+1} \delta_{a,l+1} - \sqrt{l} \delta_{a,l-1} \right) \delta_{b+d,k} \end{aligned} \quad (15)$$

we can see that F_1 is diagonal in the k index and has 2 off-diagonal lines at $l \pm 1$ and so only has $2(N_v - 1) * (N_{x,tot} - 1) = 2(N_v - 1) * (2N_x)$ non-zero elements (since $F_1|_{k=0} = 0$ and each diagonal has $N_v - 1$ terms). Similarly for F_2 , the number of nonzero terms can be found by multiplying the contributions from each index, so for a, b, c, d respectively, the count is $2(N_v - 1) * (2N_x + 1) * (\frac{N_v}{2}) * (2N_x)$, since $I_c = 0$ if c is odd and we don't include $d = 0$. These formulas are in agreement with the results from pyLIQTR where for $N_x = N_v = 4$ the number of nonzero terms for each F are 48 and 864. F_2 is also symmetric when swapping $(a, c) \leftrightarrow (b, d)$ so only half need to be calculated.

Luckily these matrices are sparse, since the number of non-zero elements grows much slower than the dimensions. To be precise, the ratio of non-zero/total entries is $\approx \mathcal{O}\left(\frac{1}{N_x N_v}\right)$ for both F_1 and F_2 .

3 Quantum Algorithm

Once we have a matrix system, we can use QLSA to solve for these coefficients. In Ameri et al., they explain that sometimes this technique loses any advantage over classical approaches and there are other approaches such as the Madelung transform to map it to the Schrodinger equation [2], but I was only going to consider using QLSA for direct simulation.

4 Results & Validation

I was not able to get block encoding to work for my matrix calculated from Carleman Linearization but if I did have those results, my hope was to plot the resource requirements for the two methods described in this paper (linearization from [2] and full Carleman for [3]). I also would run classical simulation of the equations for small test cases to verify the accuracy.

Future work on this topic could research some methods proposed in the papers I referenced comparing to lattice based methods, or the Madelung transform. I also would have liked to do more work on understanding the strengths of each different method since there likely are tradeoffs in runtime v.s. stability for example.

This is my (minimalist) [Github repository](#)

5 Full Stack Impact / Resource Implications

If I had finished this work, the full stack impact would have been mainly at the algorithm level. My hope was to compare the scalability and performance of these two methods, and possibly more, to find if they were possible to run utility scale problems already, or if more optimizations were needed.

References

- [1] Alexander Engel, Graeme Smith, and Scott E. Parker. “Quantum algorithm for the Vlasov equation”. In: *Phys. Rev. A* 100 (6 Dec. 2019), p. 062315. DOI: [10.1103/PhysRevA.100.062315](https://arxiv.org/abs/1907.09418). URL: <https://arxiv.org/abs/1907.09418>.
- [2] Abtin Ameri et al. “Quantum algorithm for the linear Vlasov equation with collisions”. In: *Phys. Rev. A* 107 (6 June 2023), p. 062412. DOI: [10.1103/PhysRevA.107.062412](https://arxiv.org/abs/2303.03450). URL: <https://arxiv.org/abs/2303.03450>.
- [3] Katharina Kormann and Anna Yurova. “A generalized Fourier–Hermite method for the Vlasov–Poisson system”. In: *BIT Numerical Mathematics* 61.3 (Sept. 2021), pp. 881–909. ISSN: 1572-9125. DOI: [10.1007/s10543-021-00853-4](https://doi.org/10.1007/s10543-021-00853-4). URL: <https://doi.org/10.1007/s10543-021-00853-4>.
- [4] Tamas Vaszary. *Carleman Linearization of Partial Differential Equations*. 2024. DOI: [10.48550/arXiv.2412.00014](https://arxiv.org/abs/2412.00014). arXiv: [2412.00014](https://arxiv.org/abs/2412.00014) [math.GM]. URL: <https://arxiv.org/abs/2412.00014>.